



다음 줄 가져오기

파일 디스크립터에서 한 줄씩 읽는 것은 너무 번거롭다.

요약: 이 활동은 파일 디스크립터에서 읽은 줄을 반환하는 함수를 프로그래밍하는 것입니다.

버전: 14.2

내용물

I. 나	목표	2
II.	공통 지침	3
III	AI 지침	5
IV	필수 항목	7
V	Readme 요구사항	9
VI	보너스 파티	10
VII.	제출 및 동료 평가	11

제1장

목표

이 활동을 통해 여러분은 유용한 기능을 컬렉션에 추가할 수 있을 뿐만 아니라 C 프로그래밍에서 중요한 개념인 정적 변수에 대해서도 배울 수 있습니다.

제2장

일반 지침

- 프로젝트는 C 언어로 작성되어야 합니다.
- 프로젝트는 반드시 표준에 따라 작성되어야 합니다. 추가 파일/함수가 있는 경우, 표준 준수 여부 검사에 포함되며, 표준 위반 시 0점을 받게 됩니다.
- 함수는 정의되지 않은 동작을 제외하고는 여기치 않게 종료되어서는 안 됩니다(세그멘테이션 오류, 버스 오류, 이중 해제 오류 등). 이러한 오류가 발생할 경우, 프로젝트는 제대로 작동하지 않는 것으로 간주되어 평가에서 0점을 받게 됩니다.
- 힙에 할당된 모든 메모리는 필요할 때 적절하게 해제되어야 합니다. 메모리 누수 용납되지 않을 것입니다.
- 주제에 필요한 경우, cc 명령어를 사용하여 -Wall, -Wextra, -Werror 플래그로 소스 파일을 필요한 출력으로 컴파일하는 Makefile을 제출해야 합니다.
또한, Makefile은 불필요한 재링크 작업을 수행해서는 안 됩니다.
- Makefile에는 최소한 \$(NAME), all, clean, fclean 규칙이 포함되어야 합니다.
답장.
- 프로젝트에 보너스 요소를 추가하려면 Makefile에 보너스 규칙을 포함해야 합니다. 이 규칙에는 프로젝트의 주요 부분에서 허용되지 않는 다양한 헤더, 라이브러리 또는 함수가 추가됩니다. 보너스 요소는 주제에서 달리 명시하지 않는 한 `\_bonus.{c/h}` 파일에 배치해야 합니다. 필수 부분과 보너스 부분의 평가는 별도로 진행됩니다.
- 프로젝트에서 libft 라이브러리를 사용하려면 해당 라이브러리의 소스 코드와 Makefile을 libft 폴더에 복사해야 합니다. 프로젝트의 Makefile은 라이브러리의 Makefile을 사용하여 라이브러리를 컴파일한 다음 프로젝트를 컴파일해야 합니다.
- 제출할 필요는 없으며 점수에 반영되지는 않지만, 프로젝트 테스트 프로그램을 작성해 보시기를 권장합니다. 이를 통해 자신의 작업과 동료의 작업을 쉽게 테스트해 볼 수 있습니다. 특히 논문 발표 시 이러한 테스트 프로그램이 매우 유용할 것입니다. 발표 시에는 본인이 작성한 테스트 프로그램이나 평가 대상 동료의 테스트 프로그램을 자유롭게 활용할 수 있습니다.
- 작업물을 지정된 Git 저장소에 제출하세요. Git 저장소에 있는 작업물만 채점됩니다. Deepthought가 채점을 담당하게 된 경우, 채점이 진행됩니다.

다음 줄 가져오기

파일 디스크립터에서 한 줄씩 읽는 것은 너무 번거롭다.

동료 평가가 완료된 후 평가가 진행됩니다. Deepthought의 채점 과정에서 오류가 발생하면 평가가 중단됩니다.

제3장

AI 지침

• 맥락

이 프로젝트는 여러분이 42세 교육의 기본 구성 요소를 발견하는 데 도움을 주기 위해 설계되었습니다.

핵심 지식과 기술을 제대로 익히려면 AI 도구와 지원 기능을 신중하게 활용하는 것이 필수적입니다.

진정한 기초 학습은 도전, 반복, 그리고 동료 학습 교류를 통한 진정한 지적 노력을 필요로 합니다.

학습 도구로서의 AI, 42시간 교육 과정의 일부로서의 AI, 그리고 고용 시장에서의 기대치로서의 AI에 대한 당사의 입장을 보다 자세히 알아보시려면 사내 인트라넷의 관련 FAQ를 참조하십시오.

• 주요 메시지

지름길 없이 탄탄한 기초를 다지세요.

기술 및 역량 개발에 진정으로 매진하세요.

진정한 오래 학습을 경험하고, 새로운 문제를 배우고 해결하는 방법을 익히세요.

배움의 과정은 결과보다 더 중요합니다.

인공지능과 관련된 위험성을 파악하고 효과적인 통제 방안을 개발하세요
흔히 발생하는 문제점을 피하기 위한 대책.

• 학습자 규칙:

- 특히 인공지능에 의존하기 전에, 말은 업무에 논리적인 사고를 적용해야 합니다.

다음 줄 가져오기

파일 디스크립터에서 한 줄씩 읽는 것은 너무 번거롭다.

- 인공지능에게 직접적인 답변을 요구해서는 안 됩니다.
- 인공지능에 대한 42가지 글로벌 접근 방식에 대해 알아보세요.

• 단계별 결과:

이 기초 단계에서 다음과 같은 결과를 얻게 될 것입니다.

- 기술 및 코딩의 기초를 제대로 다지세요.
- 이 단계에서는 인공지능이 왜 그리고 어떻게 위험할 수 있는지 알아야 합니다.

• 설명 및 예시:

- 네, 인공지능이 존재한다는 것을 알고 있습니다. 그리고 인공지능이 여러분의 프로젝트를 해결할 수 있다는 것도 알고 있습니다. 하지만 여러분은 배 우기 위해 여기에 온 것이지, 인공지능이 학습했다는 것을 증명하기 위해 온 것이 아닙니다. 인공지능이 주어진 문제를 해결할 수 있다는 것을 보 여주기 위해 여러분의 시간(그리고 우리의 시간)을 낭비하지 마세요.
- 42세에 배우는 것은 정답을 아는 것이 아니라, 정답을 찾아내는 능력을 개발하는 것입니다. 인공지능은 정답을 직접 알 려주지만, 이는 스스로 추론하는 능력을 키우는 것을 방해합니다. 그리고 추론에는 시간과 노력이 필요하며, 실패를 경 험하기도 합니다. 성공으로 가는 길은 결코 쉽지 않습니다.
- 시험 중에는 AI를 사용할 수 없다는 점을 명심하세요. 인터넷도, 스마트폰도 사용할 수 없습니다. 학습 과정에서 AI에 너 무 많이 의존했다면 금방 깨닫게 될 것입니다.
- 또래 학습은 다양한 아이디어와 접근 방식을 접하게 해 주어 대인 관계 능력과 창의적 사고 능력을 향상시켜 줍니다. 이는 단순히 챗봇과 대화하는 것보다 훨씬 더 가치 있는 경험입니다. 그러니 주저하지 말고 이야기하고, 질문하고, 함 께 배워나가세요!
- 네, 인공지능(AI)은 학습 도구로서뿐만 아니라 독립적인 주제로서도 교육 과정에 포함될 것입니다. 직접 AI 소프트웨어를 개발해 볼 기회도 주어집니다. 저희의 단계적 학습 방식에 대한 자세한 내용은 인트라넷에 게시된 문서를 참고하시기 바랍니다.

✓ 모범 사례:

새로운 아이디어가 막혔어요. 그래서 근처에 있는 사람에게 어떻게 접근했는지 물어봤죠. 10분 정도 이야기를 나누다 보 니 갑자기 깨달음이 왔어요. 이제 알겠어요.

잘못된 관행:

나는 몰래 AI를 사용해서 관찮아 보이는 코드를 복사했다. 동료 평가에서 아무것도 설명할 수 없어서 떨어졌다. AI를 사용 하지 않는 시험에서도 똑같이 막혀서 떨어졌다.

제4장

필수 항목

함수 이름: <code>get_next_line</code> 프로토타입: <code>char *get_next_line(int fd);</code>	
제출 파일: <code>get_next_line.c</code> , <code>get_next_line_utils.c</code> , <code>get_next_line.h</code> fd: 읽어들인 파일 디스크립터	
올 줄 수: 올바른 동작	
매개변수	
반환 값	NULL: 더 이상 읽을 내용이 없거나 오류가 발생했습니다.
외부 함수 읽기, 메모리 할당, 해제	
설명	텍스트에서 읽은 줄을 반환하는 함수를 작성하세요. 파일 디스크립터

• `get_next_line()` 함수를 반복적으로 호출하면(예: 루프 사용) 파일 디스크립터가 가리키는 텍스트 파일을 한 줄씩 읽을 수 있습니다.

• 여러분의 함수는 읽어들이는 줄을 반환해야 합니다.

읽을 내용이 없거나 오류가 발생하면 NULL을 반환해야 합니다.

• 함수가 파일을 읽을 때와 읽지 않을 때 모두 예상대로 작동하는지 확인하십시오.

표준 입력에서 읽어옵니다.

• 반환되는 줄에는 종료 문자(`\n`)가 포함되어야 합니다. 단, 파일의 끝에 도달했지만 파일 끝에 `\n` 문자가 없는 경우는 예외입니다.

• 헤더 파일 `get_next_line.h`에는 최소한 `get_next_line()` 함수의 프로토타입이 포함되어 있어야 합니다.

• 필요한 모든 도우미 함수를 `get_next_line_utils.c` 파일에 추가하세요.



정적 변수가 무엇인지 아는 것이 좋은 시작이 될 것입니다. 이다.

다음 줄 가져오기

파일 디스크립터에서 한 줄씩 읽는 것은 너무 번거롭다.

- `get_next_line()` 함수에서 파일을 읽어야 하므로, 이 옵션을 추가하세요.
컴파일러 호출: `-D BUFFER_SIZE=n` 이 옵션은 `read()` 함수의 버퍼 크기를 정의합니다.
버퍼 크기 값은 동료 평가자와 Moulinette가 코드 테스트를 위해 조정합니다.



이 프로젝트는 일반적인 플래그 외에도 `-D BUFFER_SIZE` 플래그를 사용하거나 사용하지 않고 컴파일할 수 있어야 합니다. 기본값은 원하는 대로 선택하시면 됩니다.

- 다음과 같이 코드를 컴파일하십시오(버퍼 크기는 예시로 42를 사용했습니다).
`cc -Wall -Wextra -Werror -D BUFFER_SIZE=42 <files>.c`
- `get_next_line()` 함수는 마지막 호출 이후 파일 디스크립터와 연결된 파일이 수정되었지만 `read()` 함수가 아직 파일의 끝에 도달하지 않은 경우 정의되지 않은 동작을 보일 수 있습니다.
- `get_next_line()` 함수는 바이너리 파일을 읽을 때 정의되지 않은 동작을 보일 수 있습니다. 하지만 원한다면 논리적인 방식으로 이 동작을 처리할 수 있습니다.



`BUFFER_SIZE` 값이 9999일 때도 함수가 제대로 작동하나요? 1일 때는요? 10000000일 때는요? 이유를 아시나요?



`get_next_line()` 함수가 호출될 때마다 가능한 한 적은 양의 데이터를 읽도록 하십시오. 새 줄을 만나면 현재 줄을 반환해야 합니다.

파일 전체를 읽은 다음 각 줄을 처리하지 마세요.

금지됨

- 이 프로젝트에서는 여러분의 `libft` 라이브러리를 사용할 수 없습니다.
- `lseek()` 함수는 사용이 금지되어 있습니다.
- 전역 변수 사용은 금지됩니다.

제5장

README 요구 사항

Git 저장소의 루트 디렉터리에 README.md 파일을 반드시 포함해야 합니다. 이 파일은 프로젝트에 익숙하지 않은 사람(동료, 직원, 채용 담당자 등)이 프로젝트의 내용, 실행 방법, 관련 정보 등을 빠르게 이해할 수 있도록 돕기 위한 것입니다.

README.md 파일에는 최소한 다음 내용이 포함되어야 합니다.

- 첫 번째 줄은 반드시 이탤릭체로 표시되어야 하며 다음과 같이 읽어야 합니다: This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]].
- 프로젝트의 목표와 내용을 명확하게 제시하는 "설명" 섹션 간략한 개요.
- 컴파일에 관한 관련 정보를 포함하는 "지침" 섹션, 설치 및/또는 실행.
- "자료" 섹션에는 해당 주제와 관련된 고전적인 참고 자료(문서, 기사, 튜토리얼 등) 목록과 함께 AI가 어떻게 사용되었는지에 대한 설명(프로젝트의 어떤 부분과 어떤 작업에 사용되었는지 명시)이 포함됩니다.

프로젝트에 따라 추가 섹션이 필요할 수 있습니다 (예: 사용 목적).
예시, 기능 목록, 기술적 선택 사항 등).

필요한 추가 사항은 아래에 명시적으로 나열됩니다.

- 본 프로젝트에 선택된 알고리즘에 대한 상세한 설명 및 타당성 검토 또한 포함되어야 합니다.



영어를 사용하는 것을 권장하지만, 캠퍼스의 주요 언어를 사용해도 됩니다.

제6장

보너스 파트

이 프로젝트는 간단하며 복잡한 보너스 기능을 지원하지 않습니다. 하지만 여러분의 창의력을 믿습니다. 필수 부분을 완료하셨다면, 이 보너스 섹션에 도전해 보시는 것도 좋습니다.

보너스 파트의 요구 사항은 다음과 같습니다.

- 하나의 정적 변수만 사용하여 `get_next_line()` 함수를 개발하십시오.
- `get_next_line()` 메서드는 여러 파일 디스크립터를 동시에 관리할 수 있습니다.
예를 들어 파일 디스크립터 3, 4, 5에서 읽는 경우, 각 파일 디스크립터의 읽기 상태를 추적하거나 다른 파일 디스크립터의 줄을 반환하지 않고도 호출할 때마다 다른 파일 디스크립터에서 읽을 수 있어야 합니다.

즉, `get_next_line()` 함수를 호출하여 파일 디스크립터 3에서 읽고, 그 다음 파일 디스크립터 4, 파일 디스크립터 5에서 읽고, 다시 파일 디스크립터 3, 파일 디스크립터 4 순으로 읽는 과정을 반복하더라도 각 파일 디스크립터의 읽기 상태를 잃지 않고 진행할 수 있다는 의미입니다.

보너스 파트 파일에 `_bonus.[c|h]` 접미사를 추가하세요.

이는 필수 제출 서류 외에도 다음 3개의 파일을 추가로 제출해야 한다는 의미입니다.

- `get_next_line_bonus.c`
- `get_next_line_bonus.h`
- `get_next_line_utils_bonus.c`



보너스 부분은 필수 부분을 완벽하게 이수했을 경우에만 평가됩니다. "완벽하다"는 것은 필수 부분이 누락 없이 완벽하게 수행되어 오류 없이 작동한다는 것을 의미합니다. 모든 필수 요건을 충족하지 못한 경우, 보너스 부분은 평가되지 않습니다.

모두,

제7장

제출 및 동료 평가

과제는 평소처럼 Git 저장소에 제출하세요. 발표 시에는 저장소 안의 내용만 평가됩니다. 파일 이름이 정확한지 다시 한번 확인하세요.



테스트 코드를 작성할 때 다음 사항을 기억하세요. 1) 버퍼 크기와 라인 크기는 매우 다른 값일 수 있습니다.

2) 파일 디스크립터는 일반 파일만 가리키는 것은 아닙니다. 꼼꼼하게 작업하고 동료들과 교차 검증하세요. 방어를 위해 다양하고 포괄적인 테스트 세트를 준비하십시오.

일단 통과하셨다면, 주저하지 말고 `get_next_line()` 함수를 `libft`에 추가하세요.

평가 과정에서 프로젝트에 대한 간단한 수정 요청이 있을 수 있습니다. 이는 사소한 동작 변경, 몇 줄의 코드 작성 또는 수정, 혹은 쉽게 추가할 수 있는 기능 구현 등을 포함할 수 있습니다.

이 단계는 모든 프로젝트에 적용되는 것은 아니지만, 평가 지침에 언급되어 있다면 반드시 준비해야 합니다.

이 단계는 프로젝트의 특정 부분에 대한 여러분의 실제 이해도를 확인하기 위한 것입니다.

수정 작업은 사용자가 선택한 개발 환경(예: 평소 사용하는 환경)에서 수행할 수 있으며, 평가의 일부로 특정 시간 제한이 지정되지 않은 한 몇 분 안에 완료될 것입니다.

예를 들어, 함수나 스크립트를 약간 수정하거나, 화면을 변경하거나, 새로운 정보를 저장하기 위해 데이터 구조를 조정하는 등의 작업을 요청받을 수 있습니다.

세부 사항(범위, 목표 등)은 평가 지침에 명시되며, 동일한 프로젝트라도 평가마다 다를 수 있습니다.